Week 1: Data Warehousing and SQL for Healthcare Data

Topics Covered:

- Introduction to Data Warehousing for healthcare data (e.g., vitals, medical records, patient history)
- SQL for creating tables, querying, and managing patient data, medical records, and health metrics

Capstone Project Milestone:

- **Objective**: Design a Data Warehouse schema to store patient data, vitals (e.g., heart rate, blood pressure), medical records, and health risk assessments.

Tasks:

1. **Design Schema**: Create tables to store patient demographics, vitals (heart rate, blood pressure, etc.), medical history, and predictive health insights.
2. **Querying Data**: Write SQL queries to monitor patient health metrics, identify abnormal vitals, and calculate risk scores for various health conditions.

Example Code:

```sql
-- Create schema for patient health data
CREATE TABLE patient_dim (
    patient_id INT PRIMARY KEY,
    patient_name VARCHAR(255),
    age INT,
    gender VARCHAR(50),
    location VARCHAR(255)
);

CREATE TABLE vitals_fact (
    vitals_id INT PRIMARY KEY,
    patient_id INT,
    heart_rate INT,  -- In beats per minute
    blood_pressure VARCHAR(10),  -- Systolic/Diastolic
    temperature DECIMAL(4, 2),  -- In Celsius
    reading_time TIMESTAMP
);

CREATE TABLE health_risk_fact (
    risk_id INT PRIMARY KEY,
    patient_id INT,
    risk_type VARCHAR(255),  -- e.g., "Heart Disease Risk"
    risk_score DECIMAL(4, 2),  -- Risk score from 0 to 1
    assessment_time TIMESTAMP
);

-- Query to calculate average heart rate by patient
SELECT patient_id, AVG(heart_rate) AS avg_heart_rate
FROM vitals_fact
GROUP BY patient_id
ORDER BY avg_heart_rate DESC;
```

Outcome: By the end of Week 1, participants will have designed a Data Warehouse schema for storing patient health data, vitals, and predictive health insights, along with SQL queries to monitor and analyze patient health metrics.

Week 2: Python for Data Collection and Preprocessing

Topics Covered:

- Python for collecting healthcare data from medical devices (e.g., wearables, IoT sensors) and electronic health records (EHRs)
- Feature engineering for predictive health models (e.g., age, heart rate, blood pressure, past medical conditions)

Capstone Project Milestone:

- **Objective**: Collect and preprocess healthcare data using Python, and generate features that can be used to predict health risks or monitor patient health in real-time.

Tasks:

1. **Data Collection**: Use APIs or streaming sources to collect real-time patient data from medical devices and EHRs.
2. **Data Preprocessing**: Clean and preprocess the data (e.g., handle missing values, normalize vitals).
3. **Feature Engineering**: Create features such as age, heart rate variability, blood pressure trends, and historical medical conditions to predict health risks.

Example Code:

```
import pandas as pd
import requests

# Collect real-time patient vitals data from a healthcare API (replace with real API)
response = requests.get("https://api.healthcare.com/vitals")
vitals_data = pd.DataFrame(response.json())

# Feature engineering: calculate heart rate variability and normalize blood pressure
vitals_data['reading_time'] = pd.to_datetime(vitals_data['reading_time'])
vitals_data['heart_rate_variability'] = vitals_data['heart_rate'].diff().fillna(0)
vitals_data['normalized_bp'] = vitals_data['blood_pressure'].apply(lambda x:
int(x.split('/')[0])) / 120  # Normalizing systolic BP

# Display the processed vitals data
print(vitals_data[['patient_id', 'heart_rate', 'heart_rate_variability',
'normalized_bp']].head())
```

Outcome: By the end of Week 2, participants will have collected and preprocessed real-time healthcare data, and generated features to be used in predictive health models.

Week 3: Real-Time Data Processing with Apache Spark and PySpark

Topics Covered:

- Using Apache Spark and PySpark for processing large-scale real-time healthcare data

- Real-time anomaly detection for patient vitals, detecting abnormal conditions or critical health risks

Capstone Project Milestone:

- **Objective**: Process real-time healthcare data using Apache Spark and PySpark to detect anomalies in patient vitals (e.g., abnormal heart rate, blood pressure spikes).

Tasks:

1. **Set Up Spark Streaming**: Configure Apache Spark for handling real-time streaming of healthcare data from wearables, sensors, and medical devices.
2. **Anomaly Detection**: Use PySpark to detect anomalies in real-time, such as abnormal heart rate, blood pressure, or temperature spikes, which could indicate health risks.

Example Code:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

# Create Spark session for streaming patient vitals data
spark = SparkSession.builder.appName("PatientMonitoring").getOrCreate()

# Read streaming data from medical devices (e.g., Kafka)
vitals_stream = spark.readStream.format("kafka").option("subscribe",
"patient_vitals").load()

# Detect anomalies: identify abnormal heart rate or blood pressure levels
anomalies = vitals_stream.filter((col("heart_rate") > 100) | (col("heart_rate") < 50))

# Write detected anomalies to console or database
query = anomalies.writeStream.outputMode("append").format("console").start()
query.awaitTermination()
```

Outcome: By the end of Week 3, participants will have implemented real-time anomaly detection using Apache Spark and PySpark to monitor and manage patient vitals in real-time, helping identify critical health risks.

Week 4: Building Predictive Health Models with Azure Databricks

Topics Covered:

- Azure Databricks for building and deploying machine learning models for predictive health insights
- Training a prediction model to forecast potential health risks (e.g., heart disease, diabetes)

Capstone Project Milestone:

- **Objective**: Build and deploy a machine learning model in Azure Databricks to predict potential health risks based on historical patient data and real-time vitals.

Tasks:

1. **Model Building**: Use Azure Databricks to train a machine learning model (e.g., logistic regression, decision trees) on historical patient data to predict potential health risks.
2. **Deploy the Model**: Deploy the model in Databricks to forecast health risks for patients based on real-time inputs (e.g., vitals, medical history).

Example Code:

```
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.feature import VectorAssembler

# Load historical patient health data
patient_df = spark.read.csv('/mnt/data/patient_health.csv', header=True,
inferSchema=True)

# Feature engineering: prepare features for the model
assembler = VectorAssembler(inputCols=["age", "heart_rate", "blood_pressure",
"temperature"], outputCol="features")
patient_df = assembler.transform(patient_df)

# Train a logistic regression model to predict heart disease risk
lr = LogisticRegression(featuresCol="features", labelCol="risk_score")
model = lr.fit(patient_df)

# Deploy the model: use it to predict health risks in real-time
real_time_vitals = spark.readStream.format("kafka").option("subscribe",
"real_time_vitals").load()
predictions = model.transform(real_time_vitals)
predictions.select("patient_id",
"prediction").writeStream.outputMode("append").format("console").start()
```

Outcome: By the end of Week 4, participants will have built and deployed a machine learning model in Azure Databricks that predicts potential health risks based on real-time vitals and patient history.

Week 5: Automating the Healthcare Monitoring System with Azure DevOps

Topics Covered:

- Automating deployment of the healthcare monitoring and prediction system with Azure DevOps
- Implementing CI/CD pipelines for deploying and monitoring healthcare data pipelines and prediction models

Capstone Project Milestone:

- **Objective**: Deploy and automate the healthcare monitoring and prediction system using Azure DevOps for continuous integration and monitoring of healthcare data pipelines and prediction models.

Tasks:

1. **Azure DevOps Pipelines**: Set up Azure DevOps pipelines to automate the deployment of the healthcare monitoring and prediction system.
2. **Monitoring and Alerts**: Set up monitoring and alerts to track anomalies in patient vitals, predict potential health risks, and detect emergencies in real-

time.

Example Code:

```yaml
# Azure DevOps pipeline YAML for deploying the healthcare monitoring system
trigger:
  - main

pool:
  vmImage: 'ubuntu-latest'

steps:
  - task: UsePythonVersion@0
    inputs:
      versionSpec: '3.x'

  - script: |
      pip install -r requirements.txt
      python deploy_healthcare_monitoring.py
    displayName: 'Deploy Healthcare Monitoring System'

  - task: AzureMonitorMetrics@0
    inputs:
      monitorName: 'HealthRiskAlerts'
      alertCriteria: 'Critical Vitals or Predicted Health Risk Detected'
```

Outcome: By the end of Week 5, participants will have deployed and automated the healthcare monitoring and prediction system using Azure DevOps. Alerts will be set up to track real-time anomalies in patient vitals and predicted health risks.

Summary of Outcomes:

1. **Week 1**: Design a Data Warehouse schema for healthcare data, such as patient demographics, vitals, and health risk assessments. Write SQL queries to monitor patient vitals and calculate risk scores for various conditions.
2. **Week 2**: Collect and preprocess real-time healthcare data using Python, including patient vitals from medical devices and EHR data. Generate features such as heart rate variability, normalized blood pressure, and historical medical conditions for predictive health models.
3. **Week 3**: Implement real-time anomaly detection using Apache Spark and PySpark to monitor patient vitals in real-time, helping detect critical health risks like abnormal heart rate or blood pressure.
4. **Week 4**: Build and deploy a machine learning model in Azure Databricks to predict potential health risks based on patient demographics, vitals, and historical data. The model will provide real-time insights into patient health, allowing for proactive intervention.
5. **Week 5**: Automate the deployment and monitoring of the healthcare monitoring and prediction system using Azure DevOps. Implement CI/CD pipelines for continuous integration, with monitoring and alerts to track health anomalies and predicted risks in real-time.

Potential Use Cases and Benefits:

1. **Patient Health Monitoring**:
    - Continuous monitoring of patients' vitals in real-time, allowing healthcare providers to detect early warning signs of deterioration and intervene quickly.
2. **Chronic Disease Management**:
    - Proactive management of chronic conditions such as diabetes, hypertension, or heart disease by predicting health risks based on vitals and historical data.
3. **Emergency Alerts**:
    - The system can alert healthcare professionals when critical health events occur, such as sudden drops in heart rate or spikes in blood pressure, allowing for timely responses.
4. **Personalized Healthcare**:
    - Machine learning models can analyze patient-specific data and provide personalized recommendations for improving health outcomes based on individual risk factors.
5. **Remote Patient Monitoring**:
    - With the integration of IoT devices and wearables, healthcare providers can remotely monitor patients' health, reducing the need for frequent in-person visits and improving care for patients in remote locations.

Technologies Used:

- **SQL**: Data warehousing and querying for healthcare data.
- **Python**: Data collection from medical devices and APIs, preprocessing, and feature engineering.
- **Apache Spark/PySpark**: Real-time data processing and anomaly detection for patient vitals.
- **Azure Databricks**: Machine learning for predictive health insights, real-time analytics, and ETL pipelines.
- **Azure DevOps**: CI/CD automation, deployment pipelines, and monitoring tools for system deployment and maintenance.
